atlántida
CIDEP
Centro de Investigación y Desarrollo de Estudios Profesionales

JSON

Lenguajes de Marcas
Profesor: Daniel López Lozano

Índice

- Qué es JSON
- Sintaxis
- Objetos
- Ficheros .json
- Validación
- Objetos JSON
- Arrays
- JSON vs XML
- Ejercicios

JSON

- **JSON** (JavaScript Object Notation - Notación de Objetos de JavaScript) es un formato ligero de intercambio de datos. Leerlo y escribirlo es simple para humanos, mientras que para las máquinas es simple interpretarlo y generarlo
- Está basado en JavaScript, de hecho JavaScript puede leer documentos JSON de forma nativa.
- Sin embargo la mayoría de lenguajes: Java, C, Python… incluyen herramientas para leer datos JSON

Sintaxis JSON

- Un fichero JSON esta compuesto por:
  - Un único objeto
  - Un array de objetos
- Centrémonos primero en representar objetos y despues cómo podemos representar arrays de objetos

Objetos JSON

- El inicio del objeto de marca con ‘{’
- El fin del objeto se marco con ‘}’
- Dentro las llaves tendremos un conjunto de 1 a N pares nombre/valor
- Cada par nombre/valor separa el nombre y el valor mediante ‘:’
- Los nombres son una cadena de texto entre comillas dobles
- Los valores podrán ser: Strings, números, otros objetos o arrays (de Strings, números, objetos o arrays)
- Cada par nombre/valor se separará por ‘,’

Objetos

- Pero… vayamos poco a poco, empecemos creando un objeto JSON válido en el que los valores sean Strings o números:

```
{
    "Nombre": "La Tierra",
    "Radio": 6378,
    "UnidadRadio": "km",
    "Gravedad": 9.8,
    "UnidadGravedad": "m/s^2"
}
```

Ficheros y validación

- Los datos JSON son escrito en documentos de texto plano con la extensión JSON
- Existen páginas webs dónde puedes validar que tus datos JSON sean válidos, como por ejemplo: https://jsonchecker.com/

```
1 {
2     "Nombre": "La Tierra",
3     "Radio": 6378,
4     "UnidadRadio": "km",
5     "Gravedad": 9.8,
6     "UnidadGravedad": "m/s^2"
7 }
```

Valid JSON!

Ejemplo de JSON válido

- Rercordemos que un fichero JSON esta compuesto por:
  - Un único objeto
  - Un array de objetos
- Por lo tanto le siguiente documentos JSON, aunque por separado sean dos objetos JSON válidos, NO sería válido

```
{
    "Nombre": "Venus",
    "Radio": 6051,
    "UnidadRadio": "km",
    "Gravedad": 8.87,
    "UnidadGravedad": "m/s^2"
}
{
    "Nombre": "La Tierra",
    "Radio": 6378,
    "UnidadRadio": "km",
    "Gravedad": 9.8,
    "UnidadGravedad": "m/s^2"
}
```

Ejemplo de JSON válido

```
1  {
2      "Nombre": "Venus",
3      "Radio": 6051,
4      "UnidadRadio": "km",
5      "Gravedad": 8.87,
6      "UnidadGravedad": "m/s^2"
7  }
8  {
9      "Nombre": "La Tierra",
10     "Radio": 6378,
11     "UnidadRadio": "km",
12     "Gravedad": 9.8,
13     "UnidadGravedad": "m/s^2"
14 }
```

```
Parse error on line 7:
...avedad": "m/s^2"} {  "Nombre": "La Tie
---------------------^
Expecting 'EOF', '}', ',', ']', got '{'
```

Arrays en JSON

- En JSON declaramos arrays con ‘[]’
- Los elementos de los arrays podrán ser: Strings, números, objetos e incluso otros arrays
- Por lo tanto el código anterior podríamos hacerlo válido insertando los dos objetos en un array

```
[
    {
        "Nombre": "Venus",
        "Radio": 6051,
        "UnidadRadio": "km",
        "Gravedad": 8.87,
        "UnidadGravedad": "m/s^2"
    },
    {
        "Nombre": "La Tierra",
        "Radio": 6378,
        "UnidadRadio": "km",
        "Gravedad": 9.8,
        "UnidadGravedad": "m/s^2"
    }
]
```

Arrays en JSON

- Los arrays no tienen que ser de un único tipo, pueden contener todo tipo de datos.
- Ejemplos de arrays válidos:

```
[1, 2, "hola"]
```

```
[1, 2.33, "hola"]
```

```
[
    1,
    2.33,
    "hola"
]
```

```
[1, 2, 3]
```

```
[
    1,
    "Adios",
    [1, 3, "Adios"],
    {
        "nombre": "Borja Molina Zea",
        "juegoFavorito":{
            "nombre": "Bloodborne",
            "fechaLanzamiento": "24/03/2015"
        }
    }
]
```

Objetos JSON

- ¿Cómo podríamos añadir los datos de La Luna a La Tierra?

```
[
    {
        "Nombre": "Venus",
        "Radio": 6051,
        "UnidadRadio": "km",
        "Gravedad": 8.87,
        "UnidadGravedad": "m/s^2"
    },
    {
        "Nombre": "La Tierra",
        "Radio": 6378,
        "UnidadRadio": "km",
        "Gravedad": 9.8,
        "UnidadGravedad": "m/s^2",
        "Satélite" : {
            "Nombre": "La Luna",
            "Radio": 1737
        }
    }
]
```

Objetos JSON

- Ahora bien… imagina que queremos añadir los datos de Marte, el cual tiene dos lunas… ¿te parece idónea la forma utilizada para La Tierra?

```
[
    {
        "Nombre": "Venus",
        "Radio": 6051,
        "UnidadRadio": "km",
        "Gravedad": 8.87,
        "UnidadGravedad": "m/s^2"
    },
    {
        "Nombre": "La Tierra",
        "Radio": 6378,
        "UnidadRadio": "km",
        "Gravedad": 9.8,
        "UnidadGravedad": "m/s^2",
        "Satélite" : {
            "Nombre": "La Luna",
            "Radio": 1737
        }
    }
]
```

Objetos JSON

```
[
    {
        "Nombre": "Venus",
        "Radio": 6051,
        "UnidadRadio": "km",
        "Gravedad": 8.87,
        "UnidadGravedad": "m/s^2"
    },
    {
        "Nombre": "La Tierra",
        "Radio": 6378,
        "UnidadRadio": "km",
        "Gravedad": 9.8,
        "UnidadGravedad": "m/s^2",
        "Satélites" : [
            {
                "Nombre": "La Luna",
                "Radio": 1737
            }
        ]
    },
```

```
    {
        "Nombre": "Marte",
        "Radio": 3389,
        "UnidadRadio": "km",
        "Gravedad": 3.7,
        "UnidadGravedad": "m/s^2",
        "Satélites" : [
            {
                "Nombre": "Fobos",
                "Radio": 11
            },
            {
                "Nombre": "Deimos",
                "Radio": 6
            }
        ]
    }
]
```

Objetos JSON

- ¿Ves adecuado repetir para cada planeta las unidades de las medidas?
- Abre el fichero ejemplo1.json y fíjate como están los datos

Usos de JSON

- Usos generales:
  - Estructurar datos de forma que resulten fácilmente interpretables por lenguajes de programación
  - Establecer un formato común para el traspaso de información entre SSII
- Algunos ejemplos de usos concretos:
  - Definir objetos y arrays en JavaScript
  - La mayoría de Web APIs ofrecen datos en formato JSON
  - Muchos programas definen sus ficheros de configuración en formato JSON, por ejemplo Visual Studio Code (Shift+Ctrl+P y busca «Default Settings»)
  - Podemos guardar datos de nuestros programas en formato JSON, por ejemplo hacer que el «save» de nuestro juego guarde los datos en el fichero sava.json

JSON vs XML

- La sintaxis JSON es más simple, lo que hace que para un mismo conjunto de datos los ficheros JSON pesen menos
- Para un mismo conjunto de datos, la interpretación por parte de un lenguajes de programación de un fichero JSON emplea, de media, menos tiempo que esos mismos datos en formato XML.
- Aunque normalmente tanto lo datos JSON como XML son generados de forma automática la sintaxis más simple JSON hace que se pueda escribir datos en JSON más rápidamente que en XML
- JSON soporta arrays XML no

JSON vs XML

- En los últimos años JSON ha desplazado en muchos ámbitos a XML
- En la programación web se puede considerar que JSON es un estándar de facto para el traspaso de datos entre webs, webs apps y web APIs
- En los programas de escritorio JSON también es utilizado como formato para los ficheros de configuración
- Sin embargo sigue habiendo áreas, como la definición interfaces gráficas, donde se sigue utilizando XML sobre JSON
- Además aunque muchos ‘datasets’ estén en JSON todavía es frecuente encontrar algunos ‘datasets’ que solo estén en formato XML

Ejercicio 1

- Transforma los datos de filosofos.xml a formatos JSON. No hace falta que sean todos los filósofos, con que lo hagas para Hegel y Nietzsche vale

Ejercicio 2

- Imagina que has programado el videojuego de plataformas. El juego tiene 3 mundos y cada mundo de 1 a 7 niveles. El protagonista puede tener de 0 a 99 vidas que se mantienen entre niveles. El protagonista puede adquirir los siguiente poderes: «bola de fuego», «bola de nieve» y «bola de rayo»
- Cuando el usuario termine de jugar queremos que cuando vuelva a entrar empiece por el nivel en el que se quedó y que conserve el número de vidas y poderes que tuviese
- Piensa como podría ser una estructura JSON válida para albergar el guardado del juego

Ejercicio 2

- Crea el fichero ejercicio2.json que tenga los siguientes datos:
  - Me quedé en el nivel 4 del mundo 2
  - Me quedé con 7 vidas
  - Adquirí los poderes de «bola de fuego» y «bola de rayo»
- ¿Crees que es seguro guardar el estado del juego un fichero de texto plano? ¿Qué solución le darías?